

BOLT: Bootstrapping-Aware Logic Resynthesis and Technology Mapping for Efficient TFHE Circuits

Bhuvnesh Chaturvedi¹  , Ayantika Chatterjee¹ , Anupam Chattopadhyay² , Debdeep Mukhopadhyay¹ 

¹Indian Institute of Technology Kharagpur, ²Nanyang Technological University Singapore

Abstract—Recent interest in fully homomorphic encryption (FHE) has motivated efforts to develop faster and more efficient homomorphic logic circuits. Currently, Torus FHE (TFHE) provides the fastest gate-level bootstrapping and enables homomorphic evaluation over encrypted bits. Prior works typically utilize standard Computer Aided Design (CAD) tools to synthesize TFHE-amenable gate-level netlists. However, the logic resynthesis and technology mapping stages of these tools are designed for reducing area and power rather than bootstrapped gates, which leads to suboptimal netlists for homomorphic execution. In this work, we introduce **BOLT**, a TFHE-amenable CAD synthesis framework that explicitly targets reductions in bootstrapping depth. **BOLT** employs a bootstrapping-aware logic resynthesis stage to rewrite sub-networks in the input netlist into functionally equivalent forms that can be evaluated with a single bootstrapping operation. Following this, our bootstrapping-aware technology mapping stage incorporates a custom simulated annealing-based scheduler to decide whether each transformed sub-network should replace its original counterpart immediately or be deferred to enable better future opportunities. Through extensive evaluation on standard benchmark circuits, **BOLT** achieves significant improvements in homomorphic evaluation time, delivering $8\times$ speedup over the current state-of-the-art and $31\times$ speedup over prior work on FHEW-like schemes.

Index Terms—Logic Resynthesis, Technology Mapping, Bootstrapping, TFHE, EDA, CAD

I. INTRODUCTION

The rapid growth of data-driven applications has created a pressing need to process large volumes of data efficiently. Cloud computing has transformed this process by allowing users and organizations to delegate their computation requirements to third-party servers. However, this convenience comes with serious privacy concerns, as clients must trust external providers to protect the confidentiality of their data. Fully Homomorphic Encryption (FHE) [13] addresses this issue due to its ability to support arbitrary computations directly over the encrypted data without decrypting it first. It further ensures that the results of these computations remain encrypted, thereby providing an end-to-end privacy solution. Due to this stronger privacy guarantee, FHE is opening up new avenues in privacy-preserving computation scenarios such as outsourced cloud computing [2], privacy-preserving machine learning [22], secure e-voting [5], and smart contracts [29].

Despite its strong privacy guarantees, the real-world adoption of FHE has been limited due to the intrinsic inefficiency of computation over FHE encrypted data. The reason is that the security of these schemes comes from the presence of *noise*, a small value that is added to the FHE ciphertexts during encryption. This noise grows with each homomorphic

computation and can lead to incorrect decryption once the noise threshold is breached. To avoid this scenario, all practical FHE schemes apply an expensive *bootstrapping* operation to reset this noise, thereby allowing further computations without the risk of incorrect decryption. Unfortunately, bootstrapping consumes a significant portion ($\approx 99\%$) of the FHE computation time, thereby dominating both the delay and the overall runtime of the FHE circuits.

Currently, the fastest scheme in terms of gate-level bootstrapping time is Torus FHE (TFHE) [6], where the operations are defined in terms of homomorphic Boolean gates. These gates have a direct correspondence to the traditional Boolean gates, as they perform the same logic as their traditional counterpart, with the difference that the operations (including bootstrapping) are performed over encrypted bits. Therefore, one can convert a standard Boolean circuit (that operates over plaintext bits) into a TFHE equivalent circuit by replacing the standard logic gates with their equivalent TFHE gates.

Motivation. Logic circuits have been a cornerstone of extensive research within the semiconductor industry. Thus, leveraging the well-established methodologies and mature tools from this domain provides an effective means to create high-performance and optimized logic circuit representations of functions over TFHE. For example, Yosys [34], an open-source logic synthesis suite, has been used in a series of works [15], [16], [24], [25], [28] to construct logic circuits over TFHE. Moreover, as TFHE gates perform the same logical operations as standard Boolean gates, one can use standard cell libraries from the semiconductor foundries along with the synthesis tools to generate optimized TFHE-amenable logic circuits.

In practice, standard CAD tools are optimized for metrics such as area and power under plaintext computation assumptions, where logic gates are inexpensive and dense packing directly translates to better performance and lower cost. On the contrary, in TFHE-based computations, each logic gate (except NOT) incurs a costly bootstrapping operation, which dominates both the delay and the overall runtime of the TFHE circuit. As a result, aggressive logic resynthesis or technology mapping choices made by conventional tools can lead to suboptimal circuits when translated to the encrypted domain. For example, standard CAD tools prefer NOR(NAND(a , b), c) over AND3(a , b , NOT(c)), despite both being functionally equivalent¹, as the former occupies less area compared to the latter. However, the latter incurs a single bootstrapping (for

¹This follows from the equivalence $\overline{(\overline{a \cdot b}) + c} \equiv \overline{a \cdot b} \cdot \bar{c} \equiv a \cdot b \cdot \bar{c}$, where $\cdot$ and $+$ denote AND and OR operations, resp. and $\bar{x}$ denote negation of x .

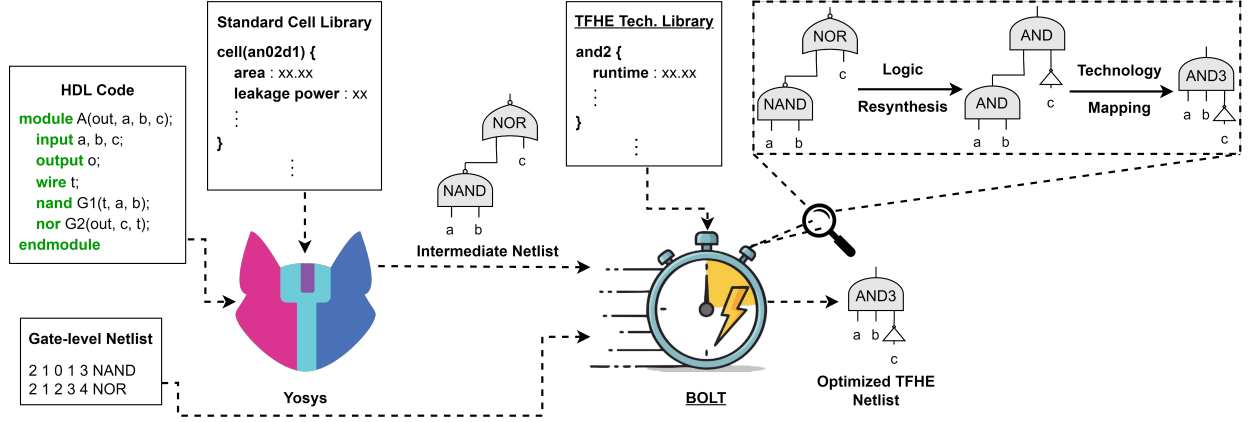


Fig. 1: **Overview of BOLT.** BOLT can process both HDL code and gate-level netlists. Input netlists are directly passed to BOLT, which applies our bootstrapping-aware logic resynthesis (Section III) and technology mapping (Section IV) algorithms on it. On the other hand, input HDL code is first synthesized using Yosys, and the generated intermediate netlist is then processed by BOLT as mentioned previously. Irrespective of the input format, the output is a TFHE-amenable netlist that can be used with the TFHE scheme [6] to process encrypted inputs. Gray = Bootstrapped gates.

AND3, since NOT is bootstrapping-free) as compared to the former, which incurs two bootstrappings (one each for NAND and NOR), and is therefore preferable in TFHE circuits.

Our solution. In this paper, we propose BOLT, an automated framework that takes as input either a program written in a Hardware Description Language (HDL), such as Verilog, or a gate-level netlist and outputs an optimized netlist tailored for computation over TFHE encrypted inputs. When the input is an HDL program, our framework first uses Yosys [34] to synthesize it into an intermediate gate-level netlist. We then apply our bootstrapping-aware logic resynthesis techniques on this intermediate netlist to further reduce the number of logic gates by packing multiple gates into a combined structure that requires a single bootstrapping.² When the input is a gate-level netlist, we directly apply our bootstrapping-aware logic resynthesis techniques on it without invoking Yosys.

Following the resynthesis stage, our bootstrapping-aware technology mapping stage integrates the transformed logic structures into the input netlist by replacing these transformations with their corresponding TFHE gates. However, immediate replacement of the logic structures can restrict opportunities for more optimal transformations in later steps. To address this, BOLT employs a Delay Reduction vs. Duplication Metric (DRDM)-based approach integrated with a simulated annealing-based scheduler. In our approach, each candidate mapping is evaluated based on its impact on bootstrapping depth³ and packing opportunities along the critical path, with the DRDM quantifying whether it yields a reduction in critical path delay relative to its structural cost. Guided by this metric, the scheduler decides whether to apply or defer each mapping, enabling more effective packing along the critical path, thereby reducing the overall delay in the generated TFHE netlists. Fig. 1 provides an overview of our framework. We now summarize our contributions:

① **Bootstrapping-aware logic resynthesis:** We incorporate logic resynthesis techniques to pack multiple bootstrapped gates into a combined structure that incurs a single bootstrap-

²In the standard CAD synthesis flow, such logic resynthesis can lead to an increase in the area footprint and are therefore ignored.

³Bootstrapping depth refers to the number of bootstrapping operations along the critical path of the TFHE circuit.

TABLE I: Summary of bootstrapped gates present in our TFHE technology library (cf. Section II-A for detailed description). **Bold** - Gate is introduced by us.

AND	OR	NAND	NOR
AND3	OR3	NAND3	NOR3
AND4	OR4	NAND4	NOR4
XOR	XNOR	XOR3	XNOR3
AND-XOR	OR-XOR	NAND-XOR	NOR-XOR
AND-XNOR	OR-XNOR	NAND-XNOR	NOR-XNOR

ping operation. When evaluated on an AES-128 circuit, this step alone leads to a reduction in the bootstrapping depth by 10.5% over its baseline implementation and by 3.3% over the current state-of-the-art implementation of [27].

② **Bootstrapping-aware technology mapping:** We propose a novel graph-based algorithm to perform custom technology mapping amenable to the optimization criteria of TFHE. Our algorithm relies on a tunable threshold, which is adjusted using our simulated annealing scheduler, to forego or proceed with the current mapping candidate. When evaluated on an AES-128 circuit, this step alone leads to a reduction in the bootstrapping depth by 22.8% over its baseline implementation and by 16.5% over the current state-of-the-art implementation of [27].

③ **BOLT:** We combine the aforementioned techniques into a synthesis framework, which we refer to as BOLT.⁴ When evaluated on an AES-128 circuit, our framework achieves a 1.25 $\times$ speedup over its baseline implementation and a 1.14 $\times$ speedup over the current state-of-the-art implementation of [27] in terms of homomorphic evaluation runtime. Our framework additionally leads to a reduction in the area-time product of the synthesized circuit, achieving a drop of 35.1% over the baseline implementation and of 9.8% over the current state-of-the-art implementation of [27].

Organization. The remainder of the paper is organized as follows. Section II provides the necessary background and related works. Section III elaborates on our logic resynthesis techniques, while Section IV explains our proposed technology mapping algorithm. Section V provides our experimental results, and Section VI concludes our paper.

⁴Our framework will be released to the community as open source upon acceptance of the paper.

II. BACKGROUND AND RELATED WORKS

In this section, we provide an overview of the TFHE library (Section II-A), a brief overview of logic resynthesis (Section II-B) and standard CAD flow (Section II-C), and a summary of related works on circuit optimization techniques for FHE schemes (Section II-D).

A. Torus FHE

A Torus FHE (TFHE) [6] ciphertext is defined as a pair $c = (a, b)$, where a is a vector whose elements are drawn uniformly from a set of integers modulo q and $b = (a \cdot s + \Delta m + e) \bmod q$. Here, $a \cdot s$ represents a dot product between a and a secret vector s , while Δm denotes an encoding of a plaintext bit m (i.e., $m \in \{0, 1\}$). Finally, e is a *small* noise value that is added to the TFHE ciphertext to ensure security.

In the original TFHE library [7], a homomorphic logic gate over two TFHE ciphertexts $c_1 = (a_1, b_1)$ and $c_2 = (a_2, b_2)$ is defined as $c_3 = (a_3, b_3) = (0, gc) \pm sf_1 \cdot (a_1, b_1) \pm sf_2 \cdot (a_2, b_2)$. Here, gc denotes a gate constant, while sf_1 and sf_2 denote integer scaling factors that differ across each gate. The order of $+$ and $-$ depends on the gate being evaluated. The underlying noise e grows due to the summation of the noises e_1 and e_2 in the input ciphertexts. Therefore, these gates are followed by a bootstrapping operation to reduce back the noise. On the contrary, a homomorphic NOT gate over a TFHE ciphertext $c = (a, b)$ is defined as $c_r = (a_r, b_r) = (-a, -b)$. Since it does not increase the underlying noise e (only its sign changes), this gate is not followed by a bootstrapping step.

The authors of [28] and [27] have shown further extensions of these basic TFHE gates to n -input gates ($n = 3, 4$), that are classified into homogenous gates such as NAND3 and OR4 and composite gates such as NAND-XOR and OR-XOR. We would like to highlight that while the homogenous gates have equivalent definitions in the standard cell libraries, the same is not true for composite gates. Therefore, using standard cell libraries for generating efficient TFHE circuits fails to leverage the benefits of these composite gates. Hence, we define our custom TFHE technology library that only consists of gates compatible with the TFHE scheme that we use in our technology mapping step. Table I lists the bootstrapped gates that are present in our technology library.

B. Logic Resynthesis

In the standard CAD domain, logic resynthesis [4] is an optimization stage aimed at improving the quality of an existing logic network while preserving its functional correctness. The process typically involves dividing the circuit's logic network into smaller sub-networks and applying Boolean optimization techniques, such as algebraic and Boolean rewriting, to simplify logic and reduce redundancy. Finally, these transformed sub-networks are mapped to standard cells from a given technology library during the technology mapping stage.

For instance, consider a sub-network like $XOR(XOR(a, b), c)$ that represents a chain of two 2-input XOR gates. One can observe that this sub-network can be directly mapped to a single $XOR3(a, b, c)$ gate. On the other hand, a sub-network

like $XNOR(XNOR(a, b), c)$, representing a chain of two 2-input XNOR gates, does not have a similar straightforward mapping. However, by rewriting the logic as $XOR(XOR(a, b), c)$ via algebraic rewriting techniques, the structure can now be mapped to an $XOR3(a, b, c)$ gate. Similarly, a sub-network like $OR(NAND(a, b), c)$ can be rewritten as $OR(OR(NOT(a), NOT(b)), c)$ via Boolean rewriting techniques such as De-Morgan's law [10], which can then be replaced with an $OR3(NOT(a), NOT(b), c)$ structure.

Hence, through such rewriting-driven transformations, logic resynthesis steps expose alternate but functionally equivalent forms of the logic network that otherwise remain hidden. When applied on TFHE-based circuits, such transformations lead to a reduction in the number of bootstrapping operations, which in turn leads to a reduction in the bootstrapping depth and the overall runtime. For example, in both the aforementioned scenarios, the original network incurs two bootstrapping operations, while the transformed network incurs only one bootstrapping operation (as NOT is bootstrapping-free).

C. CAD Flow for TFHE

A typical CAD flow begins with synthesis, which transforms a high-level HDL program into a functionally equivalent gate-level netlist suitable for physical implementation. This synthesis flow comprises several essential stages, including logic synthesis, logic resynthesis, and technology mapping. Logic synthesis generates an initial logic network from the hardware description, while logic resynthesis applies advanced optimization techniques to improve critical design metrics such as performance, power and area (PPA). Finally, technology mapping converts the optimized logic into a netlist composed of standard cells from the target technology library. Across all stages, the major design objective is to achieve favorable trade-offs among different optimization criteria, ensuring that the resulting circuit meets the overall design goal.

Standard CAD tools, although effective for initial synthesis, are not sufficient on their own to produce optimized netlists for TFHE-amenable circuits. The reason is that in a TFHE-amenable flow, the primary goal is to reduce the number of bootstrapped gates either along the critical path or in the overall circuit, thereby decreasing the latency or the overall circuit runtime. On the contrary, standard CAD flows prioritize minimizing area and power consumption.

For instance, consider a sub-network like $OR3(NOT(a), NOT(b), c)$ that is to be mapped to the gates from a given technology library. Within a TFHE-amenable flow, mapping this sub-network in its current form is preferable even though it incurs a larger area footprint since it incurs a single bootstrapping operation (to evaluate OR3, as NOT is bootstrapping free). In contrast, within the standard CAD flow, mapping this sub-network to $OR(NAND(a, b), c)$ will be preferred even though it incurs two bootstrapping operations (one each to evaluate OR and NAND) since the combined area of OR and NAND gates is less than that of OR3 and the two NOT gates. Hence, in addition to bootstrapping-aware logic resynthesis, it also becomes important to incorporate bootstrapping-aware technology mapping techniques into the standard CAD tools to tailor their flow for TFHE-based circuit optimization.

TABLE II: Summary of the related works.

Category	Related Works	Limitations	Addressing the Limitations
Boolean circuit optimizations for TFHE	[17], [24]–[28], [36]	Rely on standard CAD tools for synthesis; optimization focuses on reducing area instead of bootstrapping.	Our synthesis flow focuses on reducing bootstrapping by using bootstrapping-aware logic resynthesis and technology mapping.
Compiler-based optimizations for TFHE	[8], [14]–[16], [24]	Focuses on replacing standard blocks from high-level descriptions with TFHE-compatible blocks.	Ours focuses on optimizing these replacement blocks themselves; our work can be used in conjunction with these approaches to act as a second-level optimization.
Circuit-specific optimizations for TFHE	[18], [25], [30]–[33]	Focuses on efficient implementations of building arithmetic or cryptographic building blocks such as adders/sbox.	Ours is generic in nature; can be applied to optimize any Boolean circuit
Boolean circuit optimizations for non-TFHE schemes	[1], [21], [37]	Focus on reducing the AND-depth or multiplicative-depth of the circuits; multiplication is a costly operation in non-TFHE schemes.	Ours focuses on reducing the overall depth of the circuit; bootstrapping is required for all (barring NOT) TFHE gates to ensure correctness of the final result.

D. Related works

In this section, we elaborate on the related works from the literature. Table II provides a summary of these works.

Boolean circuit optimizations for TFHE. Existing techniques on Boolean circuit optimizations for the TFHE scheme employ either Lookup Tables (LUTs)-based or gate-based techniques to generate efficient Boolean circuits. For example, works such as Mono et al. [26], Guan et al. [17], and Yu et al. [36] leverage multi-bit LUTs, which involve efficient LUT packing via multi-value Functional Bootstrapping (FBS).⁵ However, while these approaches result in an overall reduction of the bootstrapping count, their parameter choices to ensure evaluation correctness lead to an increased per-FBS runtime. This, in turn, leads to an increase in the overall runtime of the synthesized circuit. On the contrary, works such as Matsuoka et al. [25] and FHEDA [28] propose multi-input Boolean gates that incur a single bootstrapping. Nonetheless, these works use standard CAD tools and cell libraries during synthesis, which, as highlighted in Section II-C, results in suboptimal netlists due to a difference in the optimization criteria with TFHE-amenable CAD flows. Finally, Malik et al. [24] and Singh et al. [27] independently proposed heuristics that identify mergeable bootstrapped gate chains, i.e., pairs of bootstrapped gates where the output of a gate g_1 directly drives a subsequent gate g_2 , and replace them with a single multi-input bootstrapped gate to reduce the number of bootstrapping operations. Singh et al. [27] restrict this transformation to cases where g_1 exclusively drives g_2 . Malik et al. [24] relax this constraint by additionally allowing g_1 to fan out to multiple successor gates (e.g., g_2 and g_3), provided that g_1 is mergeable with all of them.⁶ These structural restrictions, however, limit the optimization scope and can result in suboptimal netlists. On the contrary, our approach places no such restrictions on the mergeable gate chains.

Compiler-based optimizations for TFHE. There also exist toolchains such as Concrete [8] and Google Transpiler [14] that aid in converting programs written in high-level languages such as C++ or Python directly to FHE-compatible programs. Likewise, toolchains such as Romeo [16], HELM [15] and COATL [24] aid in converting programs written in HDL to

equivalent homomorphic circuits. Both classes of work operate by replacing standard blocks from high-level descriptions such as if-else statements and non-arithmetic constructs such as activation functions in machine learning with TFHE-compatible blocks such as multiplexers and LUT-based evaluations. On the contrary, our approach focuses on optimizing the FHE circuits to reduce their runtime. Therefore, our work can be used in conjunction with these approaches to optimize the building blocks themselves. For example, the c7552 circuit from the ISCAS’85 benchmark suite has 1108 TFHE gates when it is converted to an FHE-compatible program using HELM [15]. Upon further application of our framework, this count drops to 913, achieving a gate reduction of 17.6%.

Circuit-specific optimizations for TFHE. Several studies such as Stracovsky et al. [30], Trama et al. [32], and Wei et al. [33] proposed hand-tuned optimizations for cryptographic circuits such as AES [12]. These works leverage FBS with parallel implementations to accelerate the sub-operations, such as substitution box, shift row, and mix columns, of the AES cipher. In another direction, a series of works such as Matsuoka et al. [25], Kim et al. [18] and Taşel et al. [31] proposed efficient implementations of multi-input and composite gates such as half and full adders, ℓ -bit XOR gates, 3-bit Majority gates, and AND-XOR gates. The authors then leveraged these newer gates to synthesize efficient implementations of arithmetic blocks such as n -bit adders and multipliers. However, these techniques work for specific circuits only and do not generalize well. In contrast, our approach is generic and can be applied to optimize any Boolean circuit.

Boolean circuit optimizations for non-TFHE schemes. Several studies such as Aubry et al. [1], Lee et al. [21], and Yu et al. [37] have proposed logic rewriting techniques to generate optimized circuits for non-TFHE schemes such as BGV [3]. However, these works mainly focus on reducing the AND-depth of the circuits, as multiplication⁷ is a time-consuming operation in these non-TFHE schemes due to the necessity to perform costly rescaling or relinearization operations. On the other hand, our work focuses on reducing the overall depth of the circuit and not just AND-depth. The reason is that bootstrapping is not limited to one TFHE gate (such as AND) but is required for all (barring NOT) gates to ensure correctness of the final result.

⁵Native to TFHE [6] and FHEW [11], FBS is an extension of their traditional bootstrapping method, which allows evaluation of an arbitrary Boolean (or low-degree) function over the ciphertext while it is being refreshed.

⁶After merging, the functionality of g_1 is absorbed into the successor gate, making g_1 redundant. Removing g_1 eliminates one bootstrapping operation.

⁷One may note that multiplication reduces to an AND operation when performed over a $\{0, 1\}$ message space.

III. BOOTSTRAPPING-AWARE LOGIC RESYNTHESIS

In the original TFHE library [7], the authors of [6] implemented homomorphic versions of the basic 2-input logic gates. Each of these gates, except NOT, requires a bootstrapping operation to ensure the correctness of the decrypted result. Building upon this foundation, the authors of [28] introduced three-input extensions of these basic gates, allowing a chain of two 2-input gates to be replaced by a single 3-input gate, thereby almost halving the number of bootstrapping operations. Subsequently, the authors of [27] further introduced 4-input versions of the AND and OR gates, along with a set of composite gates such as AND-XOR, OR-XOR, NAND-XOR, and NOR-XOR. These enhancements enabled an even greater reduction in the number of bootstrapping operations during the synthesis of TFHE-amenable netlists.

However, the synthesis flows proposed in works such as [27] exploit these multi-input and composite gates only when a direct structural correspondence exists between a sub-network and one of the predefined gate types. For example, a sub-network is replaced by an AND3 gate only if it represents an AND-AND chain. Consequently, this approach fails to optimize cases where no direct mapping to a multi-input or composite gate is apparent. In contrast, works such as [28] leverage standard CAD tools, which perform logic resynthesis to transform sub-networks such that the resulting structures have a direct correspondence with one of the predefined gate types. Yet, these tools typically do not apply such transformations when generating TFHE-amenable netlists. This is due to differences in their underlying optimization objectives, where conventional circuit design focuses on minimizing area and power, while TFHE-based design prioritizes reducing the number of bootstrapping operations.

To bridge this gap, we introduce bootstrapping-aware logic resynthesis techniques that transform sub-networks lacking a direct mapping to a single bootstrapped gate in our TFHE technology library into functionally equivalent sub-networks that can be mapped to such gates. By rewriting logic into packing-friendly gate combinations, these techniques reduce the number of bootstrapping operations incurred along the critical path, thereby reducing the bootstrapping depth of the circuit. We now describe our proposed resynthesis techniques in detail, which are categorized into three subtypes based on the specific strategy used during the resynthesis step.

A. Via Inverter Insertion

In this category, we derive new gates from existing ones by inserting a NOT (inverter) operator into the original gate logic. The resulting gate continues to incur only a single bootstrapping operation since the added inverter does not incur a bootstrapping operation. To illustrate this approach, consider the derivation of a NAND-XNOR gate from an existing NAND-XOR gate. In a standard CAD flow, this transformation can be achieved by placing a NOT gate at the output of the NAND-XOR gate. A similar principle applies to TFHE, where the NAND-XOR gate can first be computed using the equation

$$c'_r = (\mathbf{0}, \frac{3}{8}) - 2 \cdot c_1 - 2 \cdot c_2 + 4 \cdot c_3 \quad (1)$$

(cf. [27], Lemma 3), with c_1 , c_2 and c_3 denoting the input ciphertexts, and then negated as $c_r = -c'_r$. Combining these expressions yields a direct computation for the NAND-XNOR gate in the form of the following equation

$$c_r = (\mathbf{0}, -\frac{3}{8}) + 2 \cdot c_1 + 2 \cdot c_2 - 4 \cdot c_3 \quad (2)$$

The correctness and security of the derived gate (NAND-XNOR, in this example) naturally follow from those of its base gates (NAND-XOR and NOT). Correctness remains intact since introducing the inversion logic does not increase the underlying noise, and security is preserved as the inversion is performed homomorphically on an encrypted ciphertext. The same procedure is applied to derive AND-XNOR, OR-XNOR, and NOR-XNOR gates from their respective base gates (AND-XOR, OR-XOR, and NOR-XOR). These newly derived gates are then incorporated into our TFHE technology library defined earlier (cf. Section II-A), thereby further extending its functionality.

B. Via Logic Rewriting

In this category, we apply logic resynthesis techniques to sub-networks that consist of multiple bootstrapped gates to transform them into functionally equivalent sub-networks that can be implemented using a single bootstrapped gate from our TFHE technology library. The transformation is carried out using standard algebraic and Boolean rewriting techniques. Such transformations enable a significant reduction in the number of bootstrapping operations required during homomorphic evaluation. For example, if the original sub-network consists of k logic gates, it would normally incur k bootstrapping operations. However, the transformed sub-network requires only one bootstrapping, thereby reducing the total number of bootstrappings by a factor of k .

To illustrate such transformations, let us revisit the sub-network from Section I, which comprises of a NAND-NOR chain operating on three TFHE ciphertexts c_1 , c_2 , and c_3 . The Boolean expression for this sub-network is $\overline{c_1} \cdot \overline{c_2} + c_3$, where ‘ $\cdot$ ’ and ‘ $+$ ’ represent a homomorphic AND and OR gate, resp., and $\overline{a}$ represents a negation of a . In its current form, the sub-network incurs two bootstrapping operations, one each for the homomorphic AND and OR gates. Now, by applying DeMorgan’s law [10], the logic can be rewritten as $\overline{c_1 \cdot c_2 \cdot c_3} \equiv c_1 \cdot c_2 \cdot c_3$, which can then be realized using a single AND3 gate and a NOT gate. Here, only the AND3 gate incurs bootstrapping since NOT is bootstrapping-free. The correctness and security of the rewritten structure directly follow from those of the base gates, as the transformation preserves functional equivalence and all operations continue to be performed homomorphically on encrypted ciphertexts.

Although such rewriting techniques are part of a standard CAD flow, they are typically not applied if the resulting logic violates the optimization objectives of conventional synthesis. For instance, in our case, a NAND-NOR sub-network is to be replaced with a NOT-AND3 structure. A standard CAD tool would reject this transformation, as the combined area of the AND3 and NOT gates exceeds that of the original NAND and NOR gates; that is, $\text{area}(\text{AND3}) + \text{area}(\text{NOT}) >$

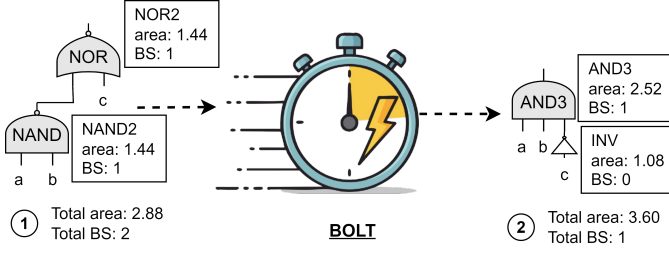


Fig. 2: Difference between the optimization criteria of standard CAD and TFHE-amenable CAD. Standard CAD prefers ① due to the lower area footprint, while TFHE-amenable CAD, including ours, prefers ② due to the lower bootstrapping count.

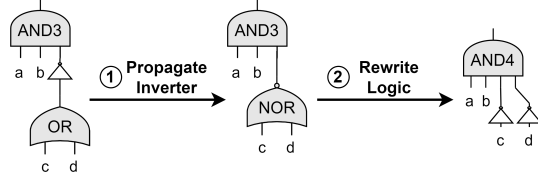


Fig. 3: Overview of our inverter propagation technique (Section III-C), which works by ① combining the inverter with the lower gate to obtain an intermediate sub-network, and ② rewriting this sub-network to obtain the final compact sub-network.

area(NAND) + area(NOR). Proceeding with this replacement would therefore increase the overall area, contradicting the traditional optimization goal of minimizing area footprint. In contrast, this transformation is desirable in a TFHE-amenable CAD flow as it reduces the number of bootstrapping operations, aligning with the optimization objective for TFHE-based circuits. Fig. 2 illustrates this distinction.

C. Via Inverter Propagation

In our logic resynthesis stage, we consider sub-networks comprising a chain of two logic gates that could be merged into a single gate, effectively halving the number of bootstrapping operations. Nonetheless, there might be some sub-networks that consist of a logic gate followed by an inverter, such as an OR followed by a NOT. In such cases, merging the two gates does not yield a direct reduction in bootstrapping operations, as both the original and the merged structures can still be evaluated with a single bootstrapping. For example, an OR–NOT combination can be replaced by a NOR gate; yet, both implementations will still incur one bootstrapping operation. Nevertheless, performing such mergers can be advantageous if the resulting structure unlocks further transformations that would otherwise not be possible.

To illustrate such transformations, consider a sub-network comprising an OR–NOT–AND3 chain, such as $\text{AND3}(a, b, \text{NOT}(\text{OR}(c, d)))$. This structure in its current form cannot be directly replaced by any single gate from our technology library via a single rewriting. To address this, we first combine the OR–NOT portion into a single NOR gate, transforming the OR–NOT–AND3 chain into a NOR–AND3 structure. In our given example, this combination will transform the structure to $\text{AND3}(a, b, \text{NOR}(c, d))$. This new structure can then be rewritten as $\text{AND4}(a, b, \text{NOT}(c), \text{NOT}(d))$ using the logic rewriting techniques discussed in Section II-B. Fig. 3 illustrates this process. The correctness and security of the transformed logic (OR–NOT–AND3 in our example) naturally follow from those of the base gates (AND4 and NOT in our

TABLE III: Summary of sub-networks that can be mapped to a single bootstrapped gate (listed in Table I) from our technology library. **Bold** - sub-network is identified by us; Underline - sub-network requires logic resynthesis before being mapped.

AND-AND	OR-OR	AND-NAND	OR-NOR
AND3-AND	OR3-OR	AND3-NAND	OR3-NOR
AND-XOR	OR-XOR	NAND-XOR	NOR-XOR
XOR-XOR	AND-AND3	OR-OR3	AND-NAND3
OR-NOR3	AND-XNOR	OR-XNOR	NAND-XNOR
NOR-XNOR	<u>XNOR-XNOR</u>	<u>XOR-XNOR</u>	<u>XNOR-XOR</u>
NAND-OR	<u>NAND-OR3</u>	<u>NAND-NOR</u>	<u>NAND-NOR3</u>
NOR-AND	<u>NOR-AND3</u>	<u>NOR-NAND</u>	<u>NOR-NAND3</u>
NAND3-OR	<u>NAND3-NOR</u>	<u>NOR3-AND</u>	<u>NOR3-NAND</u>

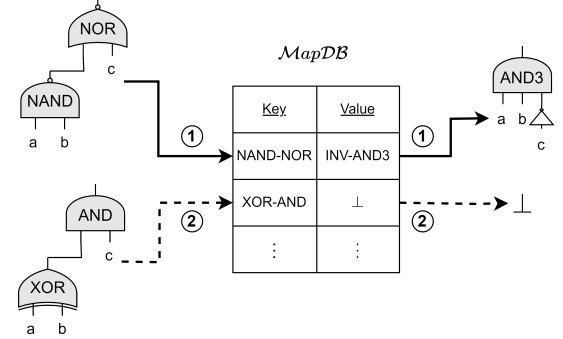


Fig. 4: Identifying valid and invalid gate combinations. We query our database *MapDB* with the identified sub-network as ‘key’. The sub-network represents a valid or invalid combination if the returned ‘value’ is ① a non-empty transformed logic or ② an empty string $\perp$, respectively.

example). The correctness remains unaffected, as the transformation preserves logical equivalence without introducing additional operations, and the security remains intact because all computations continue to be performed homomorphically on TFHE-encrypted ciphertexts. As with previous cases, such transformations are not implicitly applied in a standard CAD flow due to the difference in the optimization objectives.

In our implementation of the abovementioned resynthesis techniques, we use a well-established technique from literature known as lazy man synthesis [35]. In our approach, we first enumerate the set of sub-networks (summarized in Table III) that can be mapped to one of the homomorphic gates in our TFHE technology library (summarized in Table I) using the transformation methods discussed earlier. These combinations are then stored as key–value pairs in a mapping database *MapDB*. The input sub-network serves as the key, and its corresponding transformed logic serves as the value.

During resynthesis, when a sub-network containing a specific logic gate combination is identified, the combination is used as the key to query this database. If a value corresponding to this key exists in the database, the same (denoting the corresponding transformed logic) is returned. On the other hand, if no entry for the key is found in the database, an empty string $\perp$ is returned. For example, if the key corresponds to a sub-network with a valid gate combination such as NAND–NOR (as evident from its presence in Table III), the database returns its corresponding transformed logic, which is NOT–AND3 for our example. On the contrary, if the key corresponds to a sub-network with an invalid gate combination such as XOR–AND (as evident from its absence from Table III), the database simply returns $\perp$. Fig. 4 illustrates this process.

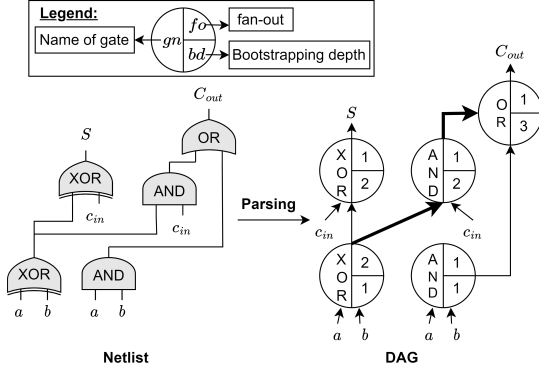


Fig. 5: Demonstration of our netlist-to-DAG conversion on a Full Adder circuit. Each node in the DAG stores the name of the gate (which is used during logic resynthesis), along with its fan-out and bootstrapping depth (both of which are used during technology mapping). Bold arrows denote the critical path in terms of bootstrapping depth.

IV. BOOTSTRAPPING-AWARE TECHNOLOGY MAPPING

In the previous section, we described how we leverage our bootstrapping-aware logic resynthesis techniques to identify a sub-network (transformed or otherwise) that can be mapped to a gate from our TFHE technology library. In this section, we describe our bootstrapping-aware technology mapping, or TechMap, stage that determines whether to proceed with the mapping or keep the structure of the sub-network intact. We first explain our parsing step, which converts the input gate-level netlist to a Directed Acyclic Graph (DAG), on which we run our logic resynthesis and TechMap stages. We then explain our TechMap algorithm in detail.

Parsing netlist to DAG. Given a gate-level netlist as input, our parsing step converts it into a DAG $\mathcal{G}$ by replacing the gates with nodes and wires with directed edges. We denote the number of nodes in this graph with $|\mathcal{G}|$. We represent the i^{th} node in $\mathcal{G}$, that corresponds to the i^{th} gate in the gate-level netlist, as $\mathcal{G}_{op_i, i_1, i_2, \dots}^i$, with $1 \leq i \leq |\mathcal{G}|$.⁸ Here, op_i denotes the output of $\mathcal{G}^i$ and $\{i_1, i_2, \dots\}$ denotes its inputs. We add an edge directed from a node $\mathcal{G}^i$ to a node $\mathcal{G}^j$, if $op_i \in \{j_1, j_2, \dots\}$, i.e., if the output of node $\mathcal{G}^i$ directly forms one of the inputs of node $\mathcal{G}^j$ or if the output of gate $\mathcal{G}^i$ drives one of the inputs of gate $\mathcal{G}^j$. We denote such node pairs $(\mathcal{G}^i, \mathcal{G}^j)$ as adjacent, represented as $\mathcal{G}^i \rightarrow \mathcal{G}^j$ going forward. We further denote $\mathcal{G}^i$ as the predecessor of $\mathcal{G}^j$, and $\mathcal{G}^j$ as the successor of $\mathcal{G}^i$. Moreover, each $\mathcal{G}^i$ stores three additional sub-fields, which refer to the name of the gate gn it corresponds to, its out-degree fo that also denotes its fan-out count, and its bootstrapping depth bd . Finally, we sort $\mathcal{G}$ in topological order, thereby arranging the nodes in levels such that gates in a level remain independent of each other and therefore can be executed in parallel. Now, for the bootstrapped gates at the first level (from the bottom), the bootstrapping depth is set as one. For the bootstrapped gates $\mathcal{G}^k$ at higher levels, the bootstrapping depth $BD(\mathcal{G}^k)$ is computed as

$$BD(\mathcal{G}^k) = \max\{BD(preds\{\mathcal{G}^k\})\} + 1 \quad (3)$$

where $preds\{\mathcal{G}^k\}$ denote the set of predecessors of $\mathcal{G}^k$ and $\max\{BD(preds\{\mathcal{G}^k\})\}$ denote the maximum value among

⁸For ease of representation, we will use “node” and “gate” interchangeably throughout the remainder of the paper, based on whether we are referring to the DAG or the gate-level netlist.

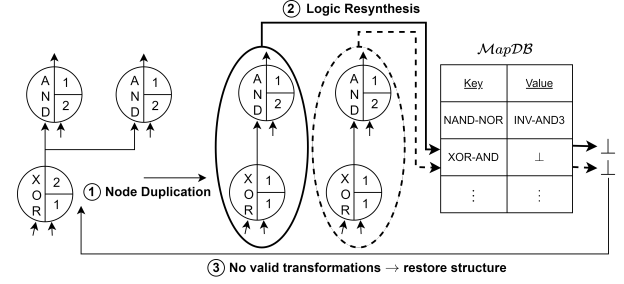


Fig. 6: **Case I.** None of the identified sub-networks (post node duplications) represents valid gate combinations. Proceeding increases the number of bootstrapping operations with no effect on the bootstrapping depth. Approach - restore structure.

the bootstrapping depths of the predecessors of $\mathcal{G}^k$. Fig. 5 summarizes this entire process on a full adder circuit.

Technology mapping or TechMap. Our technology mapping stage starts by identifying valid gate combinations (summarized in Table III) in the input netlist that can be mapped to a gate from our technology library (summarized in Table I). To do this, we take the topologically sorted DAG $\mathcal{G}$ obtained in the above step and traverse it in a top-down breadth-first search (BFS) manner. During the traversal, we consider a subgraph consisting of a pair of nodes $\mathcal{G}^i$ and $\mathcal{G}^j$ only if these nodes satisfy both these conditions:

- 1) $\mathcal{G}^i \rightarrow \mathcal{G}^j$ holds, i.e., $\mathcal{G}^i$ is adjacent to $\mathcal{G}^j$.
- 2) If a query to $MapDB$ with the key $(\mathcal{G}^i, \mathcal{G}^j)$ returns a non-empty value $\mathcal{G}^k$.

In case the second step results in an empty structure $\perp$, we leave the subgraph as it is and proceed to the next node $\mathcal{G}^{i-1}$.⁹ On the contrary, if the second step results in some non-empty structure $\mathcal{G}^k$, we do not immediately proceed with the mapping and make further checks for reasons stated later. To begin with, we check whether the out-degree of $\mathcal{G}^i$, which we denote as $\mathcal{G}^i.fo$ ¹⁰, is greater than or equal to 1. If $\mathcal{G}^i.fo$ is found to be 1, i.e., the output op_i of $\mathcal{G}^i$ only drives the input of $\mathcal{G}^j$, then we proceed with an immediate mapping. We do so by first replacing the subgraph consisting of $(\mathcal{G}^i, \mathcal{G}^j)$ with the node $\mathcal{G}^k$ in our DAG $\mathcal{G}$, after which we remove the node $\mathcal{G}^i$ from the DAG $\mathcal{G}$. The removal of $\mathcal{G}^i$ does not affect the functional correctness of the new circuit since its output is not driving the inputs of any other nodes anymore. In other words, the gate $\mathcal{G}^i$ is not making contributions towards further computations and thus acts as a dangling node. Finally, this case provides improvements in terms of both the reduction in the bootstrapping depth and the overall bootstrapping count. The reduction in the bootstrapping count comes from the removal of a (dangling) bootstrapped node. The reduction in the bootstrapping depth comes from the replacement of a chain of two bootstrapping operations (one each for $\mathcal{G}^i$ and $\mathcal{G}^j$) with a single bootstrapping operation (for $\mathcal{G}^k$).

In practical circuits, it is rare to have nodes with an outdegree of 1. In other words, there is a higher probability of the node $\mathcal{G}^i$ under consideration having an outdegree of more than 1, i.e., $\mathcal{G}^i.fo > 1$. A straightforward way to process

⁹Recall that we are traversing the DAG top-down in BFS manner, and thus, we decrement the index i to reach the next node.

¹⁰In a similar way, we denote the gate name and bootstrapping depth of $\mathcal{G}^i$ as $\mathcal{G}^i.gn$ and $\mathcal{G}^i.bd$, respectively.

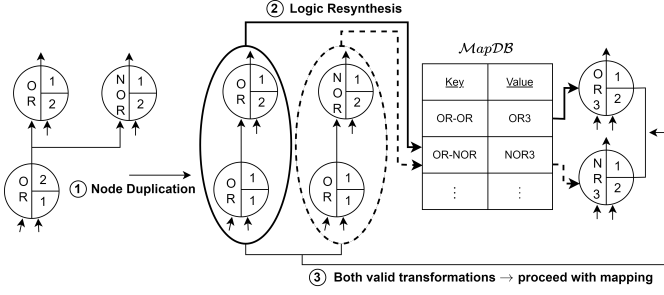


Fig. 7: **Case II.** All the identified sub-networks (post node duplications) represent valid gate combinations. Proceeding decreases both the number of bootstrapping operations and the bootstrapping depth. Approach - proceed with mapping.

these nodes is to make multiple copies of $\mathcal{G}^i$, a process we refer to as node duplication. As a result of these duplications, we end up with different node pairs such as $(\mathcal{G}^i, \mathcal{G}^j)$, $(\mathcal{G}^i, \mathcal{G}^\ell)$, and so on, with the number of such pairs being $\mathcal{G}^i.f_o$. Since the $\mathcal{G}^i.f_o$ in each of the pair is 1, each pair can now be processed as mentioned in the previous paragraph. However, not all of these pairs might have a valid entry in $MapDB$. In other words, not all pairs of nodes can be transformed and mapped to a single node, thereby leading to a node merger. Therefore, we might end up with more nodes than we began with, which in turn will lead to an increase in the number of bootstrapping operations when the synthesized netlist is evaluated homomorphically.¹¹ Hence, it becomes necessary to perform such duplications judiciously. To do so, we first identify the following three cases that arise in such a scenario.

Case I - no mergers. In this case, $\mathcal{G}^i$ cannot be merged with any of its successors. One can observe that duplication at this point results in an increase in the number of bootstrapped nodes while the bootstrapping depth of the critical path remains the same. This in turn leads to an increase in the runtime of the netlist while the delay along the critical path remains the same. Therefore, in this case, we leave the structure intact. Fig. 6 provides a summary of this case on a toy circuit.

Case II - all mergers. In this case, $\mathcal{G}^i$ can be merged with all of its successors, with the number of such mergers being equal to the outdegree $\mathcal{G}^i.f_o$ of $\mathcal{G}^i$. To perform these mergers, we first make $\mathcal{G}^i.f_o$ number of copies of $\mathcal{G}^i$ to generate $\mathcal{G}^i.f_o$ node pairs such as $(\mathcal{G}^i, \mathcal{G}^j)$, $(\mathcal{G}^i, \mathcal{G}^\ell)$, and so on. Then for each node pair, such as $(\mathcal{G}^i, \mathcal{G}^j)$, we make a query to $MapDB$ using the node pair as the key and obtain its corresponding mapping, say $\mathcal{G}^k$. We then replace the node pairs, such as $(\mathcal{G}^i, \mathcal{G}^j)$, with their corresponding mappings, such as $\mathcal{G}^k$. Once all node pairs are processed, we remove the node $\mathcal{G}^i$ from the DAG $\mathcal{G}$, since its output is not driving the inputs of any other nodes anymore, and therefore it becomes a dangling node. This approach achieves the dual reduction in both the bootstrapping depth and bootstrapping count, leading to an improvement in both the delay and total runtime of the circuit. Therefore, in this case, we proceed immediately with the mergers. Fig. 7 summarizes this case on a toy circuit.

Case III - partial mergers. In this case, $\mathcal{G}^i$ can be merged with only a subset of its successors. For example, if $\mathcal{G}^i$ is adjacent to $\mathcal{G}^j$ and $\mathcal{G}^\ell$, then we are dealing with a scenario

¹¹Node duplication does not lead to an increase in the bootstrapping depth of the circuit since the duplicated nodes remain at the same level.

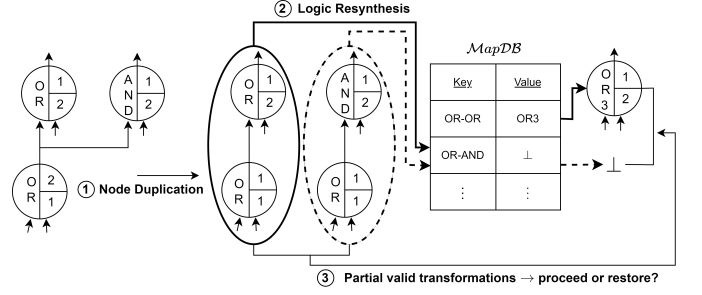


Fig. 8: **Case III.** Some of the identified sub-networks (post node duplications) represent valid gate combinations. Proceeding may lead to local optimizations but can affect global optimizations. Approach - need more information.

Algorithm 1 Valid Mergers on Critical Path

Input: $\mathcal{G}$; $MapDB$; i ;
Output: fan_out ; $valid_mergers$; $nodes_on_cp$;

```

1: function GETMergers( $\mathcal{G}$ ,  $MapDB$ ,  $i$ )
2:    $fan\_out := 0$ ;  $nodes\_on\_cp := 0$ ;
3:    $valid\_mergers := \emptyset$ ;  $wgts := \emptyset$ ;
4:   for  $j$  in  $i + 1$  to  $|\mathcal{G}|$  do
5:     if  $\mathcal{G}^i \rightarrow \mathcal{G}^j$  then
6:        $fan\_out := fan\_out + 1$ ;
7:        $wgts.append(\mathcal{G}^j.bd)$ ;
8:        $\mathcal{G}^k \leftarrow MapDB[(\mathcal{G}^i.gn, \mathcal{G}^j.gn)]$ ;
9:       if  $\mathcal{G}^k \neq \perp$  then ▷ Cond. ① ✓?
10:         $valid\_mergers.append((\mathcal{G}^i, \mathcal{G}^j) \rightarrow \mathcal{G}^k)$ ;
11:       end if
12:     end if
13:   end for
14:    $max\_bd \leftarrow maximum(wgts)$ ;
15:   for each  $vm$  in  $valid\_mergers$  do
16:     if  $vm.\mathcal{G}^j.bd == max\_bd$  then ▷ Cond. ② ✓?
17:        $nodes\_on\_cp := nodes\_on\_cp + 1$ ;
18:     end if
19:   end for
20:   return  $fan\_out$ ,  $valid\_mergers$ ,  $nodes\_on\_cp$ 
21: end function

```

where $\mathcal{G}^i$ can be merged with either $\mathcal{G}^j$ or $\mathcal{G}^\ell$, but not both. This necessitates a decision on whether to proceed with such partial mergers or forego them altogether.¹² An intuitive way to make this decision is to check whether the node combinations with valid mergers are along the critical path, and if they are, then proceed immediately with the mergers. The intuition behind this approach is that such mergers will lead to an immediate reduction in the bootstrapping depth along the critical path of the circuit. But this approach can lead to aggressive mergers along the critical path, optimizing it heavily while turning another near-critical path into the new critical path.¹³ Fig. 8 summarizes this case on a toy circuit.

From the aforementioned observations, it is apparent that

¹²The authors of [24] forego such mergers immediately, as they do not lead to a reduction in the bootstrapping count of the circuit. We later show that occasionally accepting such mergers lead to a reduction in the bootstrapping depth of the circuit, thereby improving parallel runtime.

¹³Partial mergers do not lead to a reduction in the bootstrapping count, since $\mathcal{G}^i$ still drives a few gates with which it cannot be merged. Removing $\mathcal{G}^i$ at this point will violate the functional correctness of the circuit.

while it is straightforward to deal with cases I and II, case III is tricky to handle. The reason is that, in this case, both immediate acceptance and rejection of the mergers lead to suboptimal results, with the former leading to aggressive local optimizations and the latter foregoing valid optimizations. Therefore, to make informed decisions, we propose a new metric, which we refer to as the Delay Reduction vs. Duplication metric (DRDM). For each candidate node $\mathcal{G}^i$, we compute this metric as follows (summarized in Algorithm 1). We first identify the set $\mathcal{S}$ of nodes that are adjacent to $\mathcal{G}^i$, with the cardinality of the set being $\mathcal{G}^i.f_o$. Then for each node $\mathcal{G}^j$ in $\mathcal{S}$, we check whether it satisfies both these conditions:

① If a query to $MapDB$ with the key $(\mathcal{G}^i, \mathcal{G}^j)$ returns a non-empty value $\mathcal{G}^k$.

② If the node pair $(\mathcal{G}^i, \mathcal{G}^j)$ lies on the critical or near-critical path of the circuit and therefore contributes towards its bootstrapping depth.

We denote the set of all nodes that satisfy both these conditions as $vm(\mathcal{G}^i)$, and the cardinality of this set as $nep(\mathcal{G}^i)$.

Now, once the set is identified, the DRDM metric for $\mathcal{G}^i$, denoted as $drdm(\mathcal{G}^i)$, is computed as $drdm(\mathcal{G}^i) = \frac{nep(\mathcal{G}^i)}{\mathcal{G}^i.f_o}$. Since, $0 \leq nep(\mathcal{G}^i) \leq \mathcal{G}^i.f_o$, this value remains between 0.0 and 1.0. Finally, we compare this metric value with a threshold value α , with $0.0 < \alpha < 1.0$, to check whether $drdm(\mathcal{G}^i) \geq \alpha$ or not. We proceed with the merger if the condition holds; else, we forego the same. To illustrate better, let us consider a small example where $\mathcal{G}^i$ drives the nodes $\mathcal{G}^j$ and $\mathcal{G}^l$, which makes $\mathcal{G}^i.f_o = 2$. We assume that only $\mathcal{G}^j$ lies on the critical path, which makes $nep(\mathcal{G}^i) = 1$. Based on these values, $drdm(\mathcal{G}^i) = \frac{nep(\mathcal{G}^i)}{\mathcal{G}^i.f_o} = \frac{1}{2} = 0.5$. Now, we proceed with merging $\mathcal{G}^i$ with $\mathcal{G}^j$ if $\alpha \leq 0.5$ since $drdm(\mathcal{G}^i) \geq \alpha$ holds. On the other hand, we forego this merger if $\alpha > 0.5$ since $drdm(\mathcal{G}^i) \geq \alpha$ does not hold. At this point, we highlight that we rely on DRDM only for dealing with case III defined above. For cases I and II, we proceed as per their definition without running this extra check. We now explain how the choice of α impacts the final optimization.

As stated above, we proceed with the merger only when $drdm(\mathcal{G}^i) = \frac{nep(\mathcal{G}^i)}{\mathcal{G}^i.f_o} \geq \alpha$, or if $nep(\mathcal{G}^i) \geq \alpha \times \mathcal{G}^i.f_o$. This implies that the merger is performed as long as a fraction (controlled by α) of the fan-outs contributes to the critical or near-critical paths. Based on this observation, we evaluate the following choices of α .

Choice 1 - keep α small. This results in a large number of mergers across the entire circuit since a small α is easier to achieve, even for nodes with small (1 or 2) and moderate (3 to 10) fan-outs. To illustrate better, let us assume $\alpha = 0.2$ and $\mathcal{G}^i.f_o = 4$ (moderate). In this example, it is sufficient for only one valid merger to be on the critical or near-critical path for the value of DRDM ($= \frac{1}{4} = 0.25$) to be greater than $\alpha = 0.2$, leading to a merger. Coming back, when this approach is applied to nodes with moderate fan-out, it results in reductions in not only the critical path but also in many other near-critical paths. This further ensures that optimizing only the critical path does not convert another near-critical path to the new critical path. On the other hand, exclusively favoring nodes with moderate fan-outs can eventually block nodes with small

Algorithm 2 Bootstrapping-aware technology mapping

Input: $\mathcal{G}$; $MapDB$; **Output:** $\mathcal{G}$;

```

1: function PERFORMMERGE(valid_mergers)
2:   for each  $[(\mathcal{G}^i, \mathcal{G}^j) \rightarrow \mathcal{G}^k]$  in valid_mergers do
3:     Replace sub-network  $(\mathcal{G}^i, \mathcal{G}^j)$  with  $\mathcal{G}^k$  in  $\mathcal{G}$ ;
4:   end for
5: end function
6: function TECHMAP( $\mathcal{G}$ ,  $MapDB$ ,  $i$ )
7:   Init.  $T, T_{min}, \alpha_T$ ;                                ▷ Params. for SA
8:   Init.  $\alpha, steps_{max}, \alpha_{diff}$ ;                          ▷ Params. for DRDM
9:    $steps := 0$ ;
10:  for  $i$  in  $|\mathcal{G}|$  to 1 do
11:     $steps := steps + 1$ ;
12:    if  $steps == steps_{max}$  then
13:       $\alpha := \alpha + \alpha_{diff}$ ;
14:       $steps := 0$ ;
15:    end if
16:     $fo, vm, nep \leftarrow GETMergers(\mathcal{G}, MapDB, i)$ ;
17:    if  $fo == 0$  or  $vm == \emptyset$  then                                ▷ Case I
18:      Skip  $\mathcal{G}^i$ ;
19:    else if  $|vm| == fo$  then                                          ▷ Case II
20:      PERFORMMERGE( $vm$ );
21:       $\mathcal{G} \leftarrow \mathcal{G} - \{\mathcal{G}^i\}$ ;
22:    else                                                            ▷ Case III
23:       $drdm := \frac{nep}{fo}$ ;  $\Delta := drdm - \alpha$ ;
24:      if  $drdm > \alpha$  then
25:        PERFORMMERGE( $vm$ );
26:      else
27:        Generate random  $r$  s.t.  $0 \leq r \leq 1$ ;
28:        if  $r \leq e^{\frac{\Delta}{T}}$  then
29:          PERFORMMERGE( $vm$ );
30:        else
31:          Skip  $\mathcal{G}^i$ ;
32:        end if
33:      end if
34:    end if
35:     $T := \max(T \times \alpha, T_{min})$ 
36:  end for
37:  return  $\mathcal{G}$ 
38: end function

```

fan-outs from merging later. Since the former does not lead to $nep = fo$, it does not result in the deletion of dangling nodes from the graph, thereby limiting the reduction of bootstrapping operations. Hence, a small α forces our technology mapping algorithm to prioritize the reduction of bootstrapping depth over that of bootstrapping count.

Choice 2 - keep α large. This results in a limited number of mergers across the entire circuit since a large α is difficult to achieve, especially for nodes with moderate (3 to 10) fan-outs. To illustrate better, let us assume $\alpha = 0.8$ and $\mathcal{G}^i.f_o = 8$ (moderate). In this example, it is necessary for at least seven valid mergers to be on the critical or near-critical path for the value of DRDM ($= \frac{7}{8} = 0.875$) to be greater than $\alpha = 0.8$, which is difficult to occur in practice. This limits the reduction of the bootstrapping depth along the critical and near-critical paths. On the other hand, when this approach is applied to nodes with small fan-out, it prioritizes cases where $mcp = fo$,

TABLE IV: Comparison of our proposed framework BOLT with Mono et al. [26] and Singh et al. [27]. The benchmark circuits are taken from [26]. All runtimes are in seconds.

Circuit	Mono et al. [26] Eval. time (in secs.)	Singh et al. [27] Eval. time (in secs.)	Ours Eval. time (in secs.)	Gain w.r.t. [26]	Gain w.r.t. [27]
add3	0.65	0.27	0.27	2.5×	-
add4	0.74	0.38	0.38	1.95×	-
calc	72.30	18.86	3.61	20.03×	5.23×
img-blur	10.50	7.05	2.68	3.92×	2.63×
img-ricker	12.40	7.85	3.02	4.11×	2.60×
structs3d	32.90	23.10	7.38	4.46×	3.13×
sum3d	29.10	20.91	3.55	8.20×	5.89×
strrev	70.80	17.59	2.29	30.92×	7.68×
ReLU	3.67	0.56	0.24	15.29×	2.33×
sqrt	18.60	5.61	3.74	4.97×	1.5×

which eventually leads to node deletions and in turn results in reduction of bootstrapping operations. Hence, a large α forces our technology mapping algorithm to prioritize the reduction of bootstrapping count over that of bootstrapping depth.

From the abovementioned choices, we can observe that a fixed value of α limits our TechMap algorithm to prioritize either optimization of bootstrapping depth or bootstrapping count, but not both. To overcome this limitation, we incorporate a custom simulated annealing [19] scheduler on top of our DRDM-based approach. Our scheduler starts with a high temperature T and a low value of α (to initially prioritize reduction of bootstrapping depth). It then gradually increases the value of α (to gradually prioritize reduction of bootstrapping count) as the system cools down. At every step, it computes the value of $drdm$, which is followed by the computation of $\Delta = drdm - \alpha$. Once done, it first checks whether $drdm \geq \alpha$ and proceeds with the merger if the condition holds. Otherwise, it generates a random value r , with $0.0 \leq r \leq 1.0$, and checks whether $r \leq \exp^{-\frac{\Delta}{T}}$. It proceeds with the merger if the condition holds; else, it foregoes the merger. This introduces an element of randomness, where mergers can happen probabilistically even when $drdm < \alpha$, although such mergers reduces as T reduces. Nonetheless, this results in a more balanced optimization of the input circuit that favors both the reduction of bootstrapping count and depth. Algorithm 2 summarizes our TechMap algorithm.

V. EVALUATION RESULTS

In this section, we provide the evaluation results of our proposed framework BOLT on the EPFL [23] and SCALE-MAMBA [20] benchmark circuits. We use the Yosys Open SYNthesis Suite (Yosys) [34] version 0.52 as our initial synthesis tool to convert the input HDL code to an intermediate gate-level netlist. For the TFHE executions, we chose the updated version of the original TFHE library [7] from [27], which is further modified to incorporate our gates from Section III-A. All runtimes are obtained on a system running on an Intel Xeon Silver 4210R @ 2.4 GHz CPU with 128 GB RAM and powered by Ubuntu 20.04 LTS.

In the remainder of this section, we provide comparison results between our approach and the existing techniques on Boolean circuit optimizations (cf. Section II-D for more details on these techniques). We encompass works that leverage both TFHE [6] and its predecessor FHEW [11], as these are widely used FHE schemes to efficiently evaluate Boolean circuits. We

now present a detailed discussion on the comparison between these existing approaches and our proposed methodology. We start with the LUT-based approaches of [17], [26], and [36], followed by the gate-based approaches of [27] and [28].

Comparison with Mono et al. [26]. Table IV provides the comparison results between the approaches of Mono et al. [26] and ours in terms of the runtime of the synthesized circuit. We also include the comparison result with Singh et al. [27], as they also benchmarked against the approach of [26]. From the table, we observe that our framework consistently outperforms both the original approach of [26] and the current state-of-the-art approach of [27] across all the circuits (barring add3 and add4, where we perform similarly to the approach of [27]), achieving speed-ups as large as $31\times$ and $8\times$ over [26] and [27], respectively. At this point, we like to highlight that our updated TFHE library uses the parameters from the original TFHE library [7] (cf. [27], Table 2 for exact parameters), thereby incurring a bootstrapping runtime of 23 milliseconds. On the contrary, the approach of Mono et al. [26] requires a larger parameter set (cf. [26], Table 2 for exact parameters) to support multi-value bootstrapping, which incurs a bootstrapping runtime of 290 milliseconds (value taken from [26], Table 4). This difference further arises from the fact that Mono et al. [26] proposed circuit optimizations specifically for FHEW-based schemes, which are inherently slower as compared to the TFHE scheme. Therefore, we now provide comparison results for the LUT-based approaches that use TFHE as their underlying scheme.

Comparison with Guan et al. [17] and Yu et al. [36]. Guan et al. [17] proposed an LUT-based approach to generate optimized TFHE circuits, where the input HDL design is initially converted into a gate-level netlist. This is then followed by a post-processing phase where multiple Boolean gates that share the same set of inputs are merged into a multi-input LUT. Yu et al. [36] improved this process by directly converting the input HDL into an LUT-based design. Both these approaches then evaluate the LUTs using multi-value FBS, which allows evaluation of the same or different Boolean gates so long as they share the same inputs. Finally, the authors of [36] further proposed an additional NOT-gate reduction technique to unlock more candidates for multi-value FBS. We now present the comparison results between the approaches of [17] and [36], the current state-of-the-art approach of [27], and ours on the EPFL [23] benchmark circuits.

Table V provides the comparison results between the synthesis techniques of [17], [27], [36] and ours in terms of the total number of gates present in the final synthesized netlist. From the table, we observe that our proposed technique further leads to a reduction in the bootstrapping count on circuits where Singh et al. [27] were previously performing better. The reason is that the technique of [27] only focuses on sequential gate chains where the output of a gate drives a single successor gate, thereby losing out on potential optimization candidates. On the other hand, our approach even utilizes cases where the output of a gate fans out to multiple successor gates, thereby unlocking more optimization candidates, which eventually leads to further reduction in the bootstrapping count. For other

TABLE V: Comparison of our proposed framework BOLT with Guan et al. [17], Yu et al. [36], and Singh et al. [27] on the EPFL benchmark suite [23]. The gates for all four works, including ours, only differ in their execution strategies, with [17] and [36] using multi-value FBS from [9], and [27] and ours using the traditional bootstrapping from [6].

Benchmark (EPFL)	Initial (# Gates)	Guan et al. [17] (# Gates)	Yu et al. [36] (# Gates)	Singh et al. [27] (# Gates)	Ours (# Gates)	Gain w.r.t. [27]
Adder	890	507	128	572	637	-
Barrel Shifter	3, 141	2, 496	2, 496	2, 243	1, 874	1.2×
Divider	38, 010	28, 773	13, 076	25, 402	37, 079	-
Log2	24, 670	20, 160	13, 573	16, 178	18, 486	-
Max	2, 865	2, 154	2, 066	1, 793	1, 434	1.3×
Multiplier	19, 416	14, 530	9, 957	13, 186	15, 458	-
Sine	4, 461	3, 578	2, 398	3, 004	3, 240	-
Square Root	19, 203	10, 452	8, 218	12, 604	14, 164	-
Square	14, 584	11, 968	7, 547	10, 467	11, 864	-
Coding-cavlc	685	554	542	438	361	1.3×
ALU control unit	124	97	94	82	68	1.2×
Decoder	304	291	292	304	284	1.1×
I2c controller	1, 301	1, 109	1, 029	840	663	1.3×
Int-to-float converter	237	175	170	149	121	1.3×
Memory controller	46, 442	36, 446	35, 016	27, 532	26, 750	1.1×
Priority encoder	974	833	818	681	686	-
Look-ahead XY router	218	174	126	136	138	-

TABLE VI: Comparison of our proposed framework BOLT with the LUT-based approach of Yu et al. [36] and the Gate-based approach of Singh et al. [27] on the EPFL benchmark suite [23]. All runtimes are in milliseconds (milliseconds).

Benchmark (EPFL)	Yu et al. [36] Eval. time (in milliseconds)	Singh et al. [27] Eval. time (in milliseconds)	Ours Eval. time (in milliseconds)	Gain w.r.t. [36]	Gain w.r.t. [27]
Adder	4, 096	9, 981	8, 012	-	1.25×
Barrel Shifter	79, 872	3, 338	2, 841	28.15×	1.18×
Divider	523, 040	248, 147	128, 827	4.06×	1.93×
Log2	542, 920	30, 362	29, 095	18.66×	1.05×
Max	82, 640	9, 864	8, 218	10.06×	1.2×
Multiplier	398, 280	23, 884	22, 942	17.36×	1.04×
Sine	95, 920	9, 529	8, 205	11.69×	1.16×
Square Root	328, 720	248, 355	132, 148	2.49×	1.88×
Square	301, 880	20, 085	19, 483	15.50×	1.03×
Coding-cavlc	21, 680	1, 085	1, 078	20.11×	1.01×
ALU control unit	3, 760	426	313	12.01×	1.36×
Decoder	9, 344	580	490	19.07×	1.19×
I2c controller	41, 160	1, 703	1, 303	31.59×	1.31×
Int-to-float converter	6, 800	697	589	11.55×	1.18×
Memory controller	1, 400, 640	35, 664	35, 110	39.89×	1.02×
Priority encoder	32, 720	7, 342	4, 221	7.75×	1.74×
Look-ahead XY router	5, 040	1, 252	1, 204	4.19×	1.04×

circuits, we do not perform better than Yu et al. [36] for the same reasons best stated in [27], Section 4. For brevity, LUT-based synthesis performs well for low-fanin, highly parallel sub-circuits but degrades on wide and shallow ones. This is because limited depth restricts gate mergers, leading to mostly small LUTs while larger fan-in leads to increased FBS cost. In contrast, the approaches of both [27] and ours remain effective in such settings by collapsing sequential gate chains into multi-input primitives, yielding greater reduction even when LUT grouping opportunities are limited.

Table VI provides the comparison results between the synthesis techniques of [36], [27] and ours in terms of the overall runtime of the circuit.¹⁴ From the table, we can observe that our proposed approach consistently outperforms both the original approach of [36] and that of [27] across all the

benchmark circuits, achieving speed-ups as large as 40× and 2× over [36] and [27], respectively. However, it is worth noting that the cost of FBS is significantly higher than that of the gate bootstrapping, which further grows super-linearly with the bit-length of the LUT [36]. This implies that a reduction in the number of FBS does not directly translate to a reduction in the runtime of the circuit, as is evident from Tables V and VI. For example, the divider circuit uses 3× fewer gates when synthesized with the approach of [36], yet incurs 4× the runtime when compared to our approach. Therefore, we further show comparison results with the state-of-the-art gate-based optimization methods of FHEDA [28] and Singh et al. [27] for TFHE-amenable circuits.

Comparison with FHEDA [28] and Singh et al. [27]. The gate-based approaches of both FHEDA [28] and Singh et al. [27] work by merging consecutive TFHE gates, each of which incurs a bootstrapping, into a single multi-input gate

¹⁴We do not include the runtimes of [17] as it is already outperformed by the approach of [36] (cf. Table 2 of [36] for exact results).

TABLE VII: Comparison of our proposed framework BOLT with FHEDA [28] and Singh et al. [27] on the SCALE-MAMBA benchmark suite [20]. All runtimes are in milliseconds.

Circuit	Base (# Gates)	FHEDA [28] (# Gates)	Singh et al. [27] (# Gates)	Singh et al. [27] Eval. time (in millisecs)	Ours (# Gates)	Ours Eval. time (in millisecs)	Gain in Eval. time (w.r.t. [27])
Arithmetic Functions							
64-bit add	376	312	251	7,255	256	6,408	1.13×
64-bit sub	439	375	314	6,603	323	6,534	1.01×
64-bit mult	13,675	11,722	9,830	22,156	9,893	19,431	1.14×
64-bit div	29,926	21,984	17,581	475,238	17,667	430,600	1.11×
64-bit negation	190	190	189	3,790	189	1,314	2.89×
Mod add 512	9,720	9,720	9,210	110,983	8,696	72,188	1.54×
Floating-point Operations							
FP-add	15,637	14,897	11,787	48,228	11,440	42,512	1.14×
FP-mult	44,899	39,174	33,821	54,171	33,957	50,500	1.07×
FP-lt	1,536	1,527	1,463	8,929	1,457	7,240	1.23×
FP-f2i	3,932	3,880	3,107	16,677	2,991	15,377	1.09×
FP-i2f	7,136	6,899	5,263	26,602	5,158	23,225	1.15×
floor	1,613	1,507	1,245	12,319	1,212	11,633	1.06×
ceil	1,618	1,513	1,251	12,662	1,219	11,672	1.09×
Cryptographic Functions							
AES-128	36,663	30,614	29,014	49,714	29,706	43,790	1.14×
AES-192	41,565	34,665	32,873	52,987	33,560	48,534	1.09×
AES-256	50,666	42,246	40,038	71,780	41,034	59,473	1.21×

such that this new structure incurs a single bootstrapping. These new structures are then incorporated into their standard cell libraries, which are then used either in conjunction with standard CAD tools such as Cadence Genus or in-house custom synthesis frameworks. Nonetheless, irrespective of the underlying technique, the primary optimization criterion of both these approaches is to reduce the number of bootstrapping gates in the synthesized netlist. The reason is that these works assume that the synthesized netlist will be executed in a serial fashion, and thus the reduction of the number of bootstrapped gates will translate directly into the reduction of the runtime. On the other hand, our approach focuses on optimizing the bootstrapping depth of the synthesized netlist. This in turn helps us in leveraging the multi-threaded setup of the modern systems to achieve speed-ups in runtime by evaluating the netlist in parallel. On the contrary, this may also lead to an increase in the number of bootstrapped gates in the synthesized netlist when compared with the aforementioned approaches.

We now report our results for our synthesized netlists over both area (number of gates in the netlist) and time (runtime of the netlist when executed in parallel). Although the approaches of both [28] and [27] assume a serial execution of the TFHE gates, we evaluate the netlists synthesized using their approaches via a parallel execution of the TFHE gates to ensure a fair comparison. We demonstrate our results on the SCALE-MAMBA [20] benchmark circuits, which are further categorized into Arithmetic Functions, Floating-point Operations, and Cryptographic Functions. We report our runtimes in milliseconds to preserve the granularity of these values.

Table VII provides the comparison results between the approaches of [27], [28], and ours. From the table, we observe that our approach outperforms the current state-of-the-art approach of [27] in terms of runtime on every benchmark circuit. The reason is that our approach leads to a further reduction in the bootstrapping depth of the synthesized circuit when compared to the approach of [27]. For example, our approach reduces the bootstrapping depth of the 64-bit negation circuit from its initial value of 65 to 24, an improvement of 2.71×, while the approach of [27] reduces the same to 64. Due to this, we observe an improvement in the runtime by a factor

TABLE VIII: Comparison of our proposed framework BOLT with Singh et al. [27] in terms of area-time product (AT Product) on the SCALE-MAMBA benchmark suite [20]. All AT Products are computed as the number of gates times the runtime of the circuit, with the unit being gate-milliseconds (gate-ms).

SCALE-MAMBA Circuits	Singh et al. [27] AT Product (in gate-ms)	Ours AT Product (in gate-ms)	Reduction in AT Product (in %)
Arithmetic Functions			
64-bit add	1,821,005	1,640,448	9.9%
64-bit sub	2,073,342	2,110,482	−1.8%
64-bit mult	217,793,480	192,230,883	11.7%
64-bit div	8,355,159,278	7,607,410,200	8.9%
64-bit negation	716,310	248,346	65.3%
Mod add 512	1,022,153,430	627,746,848	38.6%
Floating-point Operations			
FP-add	568,463,436	486,337,280	14.5%
FP-mult	1,832,117,391	1,714,828,500	6.4%
FP-lt	13,063,127	10,548,680	19.3%
FP-f2i	51,815,439	45,992,607	11.2%
FP-i2f	140,006,326	119,794,550	14.4%
floor	15,337,155	14,099,196	8.1%
ceil	15,840,162	14,228,168	10.2%
Cryptographic Functions			
AES-128	1,442,401,996	1,300,825,740	9.8%
AES-192	1,741,841,651	1,628,801,040	6.5%
AES-256	2,873,927,640	2,440,415,082	15.1%

of 2.89× over that of [27], although the area (gate count) for both approaches remains the same at 189. We further observe that for certain circuits, our approach even leads to a reduction in the area when compared to [27]. This is due to the added utilization of sequential gate chains, where the output of a gate fans out to multiple successor gates. This relaxation unlocks more optimization candidates when compared to the approach of [27], which considers only those gate chains where the output of a gate drives a single successor gate.

Comparison with Singh et al. [27] w.r.t. area-time product.

We now report the area-time product (AT Product) values on the SCALE-MAMBA [20] and EPFL [23] benchmark circuit suites when these circuits are synthesized using the current state-of-the-art approach of Singh et al. [27] and ours.¹⁵ We report these values to capture the trade-off between the area and runtime optimizations when using the approach of [27]

¹⁵We do not compare with the approach of [28] as it has already been outperformed by the approach of [27] (cf. Table 5 of [27] for exact results).

TABLE IX: Comparison of our proposed framework BOLT with Singh et al. [27] in terms of area-time product (AT Product) on the EPFL benchmark suite [23]. All AT Products are computed as the number of gates times the runtime of the circuit, with the unit being gate-milliseconds (gate-ms).

EPFL Circuits	Singh et al. [27] AT Product (in gate-ms)	Ours AT Product (in gate-ms)	Reduction in AT Product (in %)
Adder	5,709,132	5,103,644	10.6%
Barrel Shifter	7,487,134	5,324,034	28.9%
Divider	6,303,430,094	4,776,776,333	24.2%
Log2	491,196,436	537,850,170	-9.5%
Max	17,686,152	11,784,612	33.4%
Multiplier	314,934,424	354,637,436	-12.6%
Sine	28,625,116	26,584,200	7.1%
Square Root	3,130,266,420	1,871,744,272	40.2%
Square	210,229,695	231,146,312	-9.9%
Coding-cavlc	475,230	389,158	18.1%
ALU control unit	34,932	21,284	39.1%
Decoder	176,320	139,160	21.1%
I2c controller	1,430,520	863,889	39.6%
Int-to-float converter	103,853	71,269	31.4%
Memory controller	981,901,248	939,192,500	4.4%
Priority encoder	4,999,902	2,895,606	42.1%
Look-ahead XY router	170,272	166,152	2.4%

(which prioritizes area over runtime) and our approach (which prioritizes runtime over area). Tables VIII and IX provide the comparison results, in gate-milliseconds, for the SCALE-MAMBA and EPFL benchmark suites, respectively. Across both benchmarks, we observe that our approach outperforms the approach of [27] in terms of AT Product over most of the circuits (15 out of 16 SCALE-MAMBA circuits and 14 out of 17 EPFL circuits). For the circuits where the AT Product is higher, the % increase remain between 2% to 12%, while for circuits where the AT Product is lower, the % decrease reaches as high as 35% to 40%. These results highlight that, in general, the improvement in runtime outweighs the worsening of the area when the netlist is synthesized using our approach. Moreover, for a few circuits, our approach leads to an improvement in terms of both area and runtime for the same reasons stated earlier.

VI. CONCLUSION

In this paper, we introduced a TFHE-amenable synthesis framework named BOLT. Our framework incorporates bootstrapping-aware logic resynthesis and technology mapping techniques to reduce the bootstrapping depth of the synthesized circuits. By transforming sub-networks into structures that can be evaluated with a single bootstrapping and using a simulated-annealing-based scheduler to judiciously apply these mappings, our approach avoids the suboptimal choices made by conventional area-driven synthesis flows. Our approach led to significant improvements in homomorphic evaluation efficiency, achieving an $8\times$ reduction in end-to-end runtime over the state of the art, while also providing up to $31\times$ improvement compared to prior FHEW-like approaches. These results highlight that logic resynthesis techniques from the traditional CAD domain can be made bootstrapping-aware and, when adapted appropriately, can substantially improve the performance of TFHE-based circuits. As a future direction, we aim to investigate whether analogous adaptations of LUT-based optimization methods from the standard CAD domain can yield similar benefits for LUT-based FHE synthesis flows.

REFERENCES

- [1] P. Aubry, S. Carpov, and R. Sirdey, “Faster homomorphic encryption is not enough: Improved heuristic for multiplicative depth minimization of boolean circuits,” *Cryptographers’ Track at the RSA Conference*, pp. 345–363, 2020.
- [2] R. Awadallah and A. Samsudin, “Homomorphic encryption for cloud computing and its challenges,” *2020 IEEE 7th International Conference on Engineering Technologies and Applied Sciences (ICETAS)*, pp. 1–6, 2020.
- [3] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, “(leveled) fully homomorphic encryption without bootstrapping,” *ACM Transactions on Computation Theory (TOCT)*, vol. 6, no. 3, pp. 1–36, 2014.
- [4] K.-C. Chen, Y. Matsunaga, S. Muroga, and M. Fujita, “A resynthesis approach for network optimization,” *Proceedings of the 28th ACM/IEEE Design Automation Conference*, pp. 458–463, 1991.
- [5] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “A homomorphic lwe based e-voting scheme,” *Post-Quantum Cryptography: 7th International Workshop, PQCrypto 2016*, pp. 245–265, 2016.
- [6] —, “Tfhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [7] —, “TFHE: Fast fully homomorphic encryption library,” August 2016. [Online]. Available: <https://tfhe.github.io/tfhe/>
- [8] I. Chillotti, M. Joye, D. Ligier, J.-B. Orfila, and S. Tap, “Concrete: Concrete operates on ciphertexts rapidly by extending tfhe,” *WAHC 2020-8th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2020.
- [9] I. Chillotti, M. Joye, and P. Paillier, “Programmable bootstrapping enables efficient homomorphic inference of deep neural networks,” *Cyber Security Cryptography and Machine Learning: 5th International Symposium, CSCML 2021*, pp. 1–19, 2021.
- [10] A. De Morgan, *Formal logic: or, the calculus of inference, necessary and probable*. Taylor and Walton, 1847.
- [11] L. Ducas and D. Micciancio, “FHEw: bootstrapping homomorphic encryption in less than a second,” *Annual international conference on the theory and applications of cryptographic techniques*, pp. 617–640, 2015.
- [12] P. FIPS, “197. advanced encryption standard (aes), national institute of standards and technology, us department of commerce,” 2001.
- [13] C. Gentry, “Fully homomorphic encryption using ideal lattices,” *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pp. 169–178, 2009.
- [14] S. Gorantala, R. Springer, S. Purser-Haskell, W. Lam, R. Wilson, A. Ali, E. P. Astor, I. Zukerman, S. Ruth, C. Dibak et al., “A general purpose transpiler for fully homomorphic encryption,” *arXiv preprint arXiv:2106.07893*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.07893>
- [15] C. Gouert, D. Mouris, and N. G. Tsoutsos, “Helm: Navigating homomorphic encryption through gates and lookup tables,” *IEEE Transactions on Information Forensics and Security*, 2025.
- [16] C. Gouert and N. G. Tsoutsos, “Romeo: Conversion and evaluation of hdl designs in the encrypted domain,” *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2020.
- [17] Z. Guan, R. Mao, Q. Zhang, Z. Zhang, Z. Zhao, and S. Bian, “Autohog: Automating homomorphic gate design for large-scale logic circuit evaluation,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 7, pp. 1971–1983, 2024.
- [18] D. Kim, S. Kim, S. Lee, and D.-J. Shin, “Low-latency fully homomorphic arithmetic using parallel prefix group circuit with primitive gate bootstrapping,” *Cryptology ePrint Archive*, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2150>
- [19] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, “Optimization by simulated annealing,” *science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [20] KULeuven-COSIC, “Scale-mamba.” [Online]. Available: <https://github.com/KULeuven-COSIC/SCALE-MAMBA>
- [21] D. Lee, W. Lee, H. Oh, and K. Yi, “Optimizing homomorphic evaluation circuits by program synthesis and term rewriting,” *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 503–518, 2020.
- [22] D. C.-T. Lo, Y. Shi, H. Shahriar, B. Deng, X. Zhang, and M.-L. Chen, “Practical considerations of fully homomorphic encryption in privacy-preserving machine learning,” *2024 IEEE International Conference on Big Data (BigData)*, pp. 6330–6335, 2024.
- [23] lsils, “The epfl combinational benchmark suite.” [Online]. Available: <https://github.com/lsils/benchmarks>
- [24] R. Malik, V. Paranjape, and M. Kulkarni, “Circuit optimization using arithmetic table lookups,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 301–323, 2025.

- [25] K. Matsuoka, Y. Hoshizuki, T. Sato, and S. Bian, "Towards better standard cell library: Optimizing compound logic gates for tfhe," *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pp. 63–68, 2021.
- [26] J. Mono, K. Kluczniak, and T. Güneysu, "Improved circuit synthesis with multi-value bootstrapping for fhew-like schemes," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 4, pp. 633–656, 2024.
- [27] A. Singh, A. Chatterjee, A. Chattopadhyay, and D. Mukhopadhyay, "An efficient circuit synthesis framework for tfhe via convex sub-graph optimization," *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, p. 13–29, 2025.
- [28] A. Singh, S. Das, A. Chakraborty, R. Sadhukhan, A. Chatterjee, and D. Mukhopadhyay, "Fheda: Efficient circuit synthesis with reduced bootstrapping for torus fhe," *IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pp. 841–859, 2024.
- [29] R. Solomon, R. Weber, and G. Almashaqbeh, "smartfhe: Privacy-preserving smart contracts from fully homomorphic encryption," *2023 IEEE 8th European symposium on security and privacy (euroS&p)*, pp. 309–331, 2023.
- [30] R. Stracovsky, R. A. Mahdavi, and F. Kerschbaum, "Faster evaluation of aes using tfhe," *Poster Session, FHE. Org*, 2022.
- [31] F. S. Taşel and A. N. Saran, "Improved arithmetic efficiency in tfhe through gate-level optimizations," *The Journal of Supercomputing*, vol. 81, no. 18, p. 1633, 2025.
- [32] D. Trama, P.-E. Clet, A. Boudguiga, and R. Sirdey, "A homomorphic aes evaluation in less than 30 seconds by means of tfhe," *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pp. 79–90, 2023.
- [33] B. Wei, R. Wang, Z. Li, Q. Liu, and X. Lu, "Fregata: Faster homomorphic evaluation of aes via tfhe," *International Conference on Information Security*, pp. 392–412, 2023.
- [34] C. Wolf, "Yosys open synthesis suite," 2016. [Online]. Available: <https://github.com/YosysHQ/yosys>
- [35] W. Yang, L. Wang, and A. Mishchenko, "Lazy man's logic synthesis," *Proceedings of the International Conference on Computer-Aided Design*, pp. 597–604, 2012.
- [36] M. Yu, S. Carpov, A. Tempia Calvino, and G. De Micheli, "On the synthesis of high-performance homomorphic boolean circuits," *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pp. 51–63, 2023.
- [37] M. Yu and G. De Micheli, "Faster homomorphic operations and beyond: Expediting homomorphic computation via boolean circuit optimization," *Journal of Cryptology*, vol. 39, no. 1, p. 6, 2026.